Milestone 2: World Happiness Report

COM-480 Data Visualization | Dafer Yassine, Ali El Bouiri, Mohamed Chafai El Alaoui, Dafer Mohamed Amine

1. Project Goal

We want to build an interactive website that lets people explore the World Happiness Report data from 2015 to 2019. The idea is to walk the user through three simple questions: where are the happiest countries? What factors make them happy? And when did things change? We are targeting a general audience so the design should feel more like a data journalism piece than a research dashboard.

2. Visualizations

Section 1: "Where is happiness?" (Choropleth Map)

The first thing the user sees after the landing page is a world map colored by happiness score. We use a red-to-green color scale (red = low, green = high). There is a year slider at the top so you can switch between 2015 and 2019. When you hover over a country, a tooltip shows the name, region, score and rank. Countries with no data stay gray.

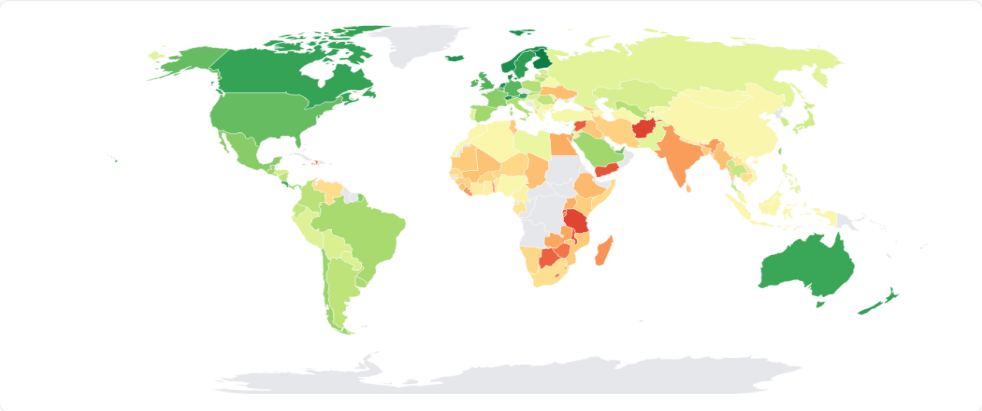

Figure 1: Choropleth map showing happiness scores for 2019

Section 2: "What drives happiness?" (Stacked Bar Chart)

This section shows a stacked bar chart that breaks down the happiness score into its six factors (GDP, social support, health, freedom, trust and generosity). By default it shows averages per region. A dropdown menu lets the user pick a specific region to see the breakdown for each country in it.

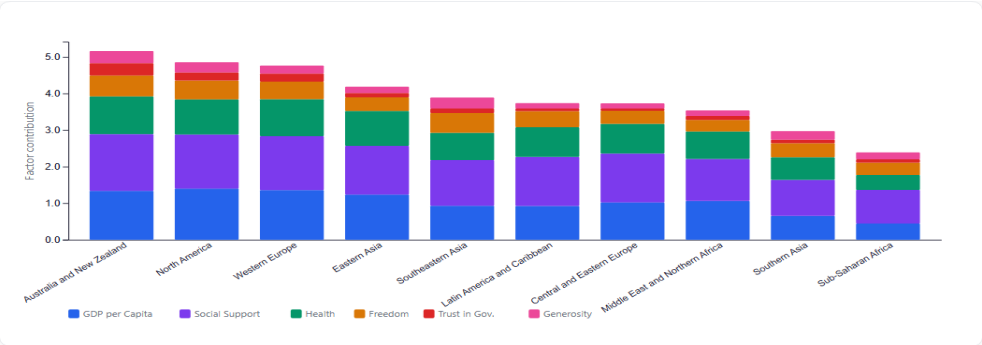

Figure 2: Factor breakdown by region (2019 averages)

Section 3: "When did things change?" (Trend Lines)

A line chart showing the trajectory of every country that appears in all 5 years (141 countries). Most lines are in light gray. The top 5 most improved and top 5 most declined countries are highlighted in green and red. Hovering any gray line reveals which country it is and how much the score changed.

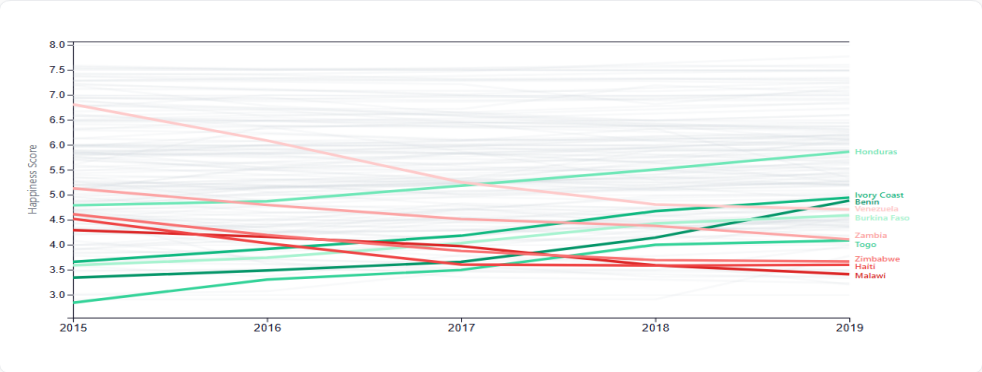

Figure 3: Happiness score trends 2015-2019 with highlighted extremes

3. Tools and Lectures

Visualization	Tools	Lectures
Choropleth world map	D3.js v7, TopoJSON, d3-geo (Natural Earth)	Maps, Practical Maps, Interactions
Year slider	HTML range input + D3 transitions	Interactions, D3.js
Stacked bar chart	D3.js (scaleBand, scaleLinear)	Tabular Data, Marks and Channels, D3.js
Region dropdown	HTML select + D3 data join/update	Interactions, D3.js
Multi-line trend chart	D3.js (d3.line, scaleLinear)	Temporal Data, Interactions, Perception
Tooltips	Custom div positioned with D3 mouse events	Interactions
Scrolling navigation	IntersectionObserver API, CSS sticky	Storytelling, Design
Data pipeline	Python + pandas (CSV to JSON)	Tabular Data

4. Core vs. Extra Features

Core (MVP)

These are the features we need for the project to make sense:

- Choropleth map with a year slider and tooltips on hover (answers "where?")
- Stacked bar chart showing factor contributions per region, with a dropdown to explore individual regions (answers "what?")
- Trend line chart with highlighted countries that improved or declined the most (answers "when?")
- Three-section layout with a sticky navigation bar so the user follows the narrative
- Data pipeline that merges the 5 CSV files into one clean JSON

Extra (nice to have)

We would like to add these if we have time but the project still works without them:

- Gapminder-style bubble chart with animated year transitions (GDP vs. score, bubble = population)
- Country comparison panel to put 2-3 countries side by side with radar charts
- Scrollama transitions for smoother animations between sections
- Sparklines in tooltips so the map tooltip also shows a tiny 5-year trend
- Search bar to find and highlight a country across all views

5. Prototype

We have a working prototype in the repo (index.html at the root). It already has all three core sections with interactive controls. You can run it with `python -m http.server` and open it in the browser.

What is working so far:

- Map with 177 countries, color scale and year slider
- Factor chart with 10 regions and per-country drill-down
- Trend chart with 141 country lines, 10 highlighted
- Sticky nav bar and tooltips on everything

Stack: plain HTML/CSS/JS, D3.js v7 and TopoJSON loaded from CDN. Data cleaned with Python/pandas. No build step needed.